

```
#!/usr/bin/env python3
```

```
"""
```

```
 국뽕유튜브 자동화 서버 (Render.com 배포용)
```

```
=====
```

```
Make.com에서 트리거 → TTS 생성 → 영상 편집 → YouTube 업로드
```

```
환경변수 (Render.com Dashboard에서 설정):
```

```
OPENAI_API_KEY=sk-...
```

```
PEXELS_API_KEY=...
```

```
TELEGRAM_BOT_TOKEN=...
```

```
TELEGRAM_CHAT_ID=...
```

```
SPREADSHEET_ID=1AlUxmQMcAt_CqW3MSiQ1eVav5XLG_wj9CEyJ-DLqH-Y
```

```
"""
```

```
import os
```

```
import json
```

```
import time
```

```
import re
```

```
import logging
```

```
import subprocess
```

```
import threading
```

```
from pathlib import Path
```

```
from datetime import datetime
```

```
from flask import Flask, request, jsonify
```

```
import requests
```

```
# =====
```

```
# 설정
```

```
# =====
```

```
app = Flask(__name__)
```

```
BASE_DIR = Path("/tmp/yt_auto")
```

```
AUDIO_DIR = BASE_DIR / "audio"
```

```
IMAGE_DIR = BASE_DIR / "images"
```

```
VIDEO_DIR = BASE_DIR / "video"
```

```
THUMB_DIR = BASE_DIR / "thumbnails"
```

```
for d in [AUDIO_DIR, IMAGE_DIR, VIDEO_DIR, THUMB_DIR]:
```

```
    d.mkdir(parents=True, exist_ok=True)
```

```
logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s] %(mes
```

```
logger = logging.getLogger(__name__)
```

```
# 환경변수
```

```

OPENAI_KEY = os.getenv("OPENAI_API_KEY", "")
PEXELS_KEY = os.getenv("PEXELS_API_KEY", "")
TG_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
TG_CHAT = os.getenv("TELEGRAM_CHAT_ID", "")
SHEET_ID = os.getenv("SPREADSHEET_ID", "1AlUxmQMcAt_CqW3MSiQ1eVav5XLG_wj9CEyJ-DLq

```

```

pipeline_status = {"running": False, "last_run": None, "last_result": None}

```

```

# =====
# 유틸리티
# =====

```

```

def notify(msg):
    """Telegram 알림"""
    if TG_TOKEN and TG_CHAT:
        try:
            requests.post(
                f"https://api.telegram.org/bot{TG_TOKEN}/sendMessage",
                json={"chat_id": TG_CHAT, "text": msg, "parse_mode": "HTML"},
                timeout=10
            )
        except:
            pass
    logger.info(f"[알림] {msg}")

```

```

def openai_request(endpoint, payload, timeout=120):
    """OpenAI API 호출 헬퍼"""
    headers = {"Authorization": f"Bearer {OPENAI_KEY}", "Content-Type": "applicat
    resp = requests.post(f"https://api.openai.com/v1/{endpoint}", headers=headers
    resp.raise_for_status()
    return resp

```

```

# =====
# 모듈 1: TTS 음성 생성
# =====

```

```

def generate_tts(narration: str, session_id: str) -> Path:
    """OpenAI TTS로 나레이션 음성 생성"""
    logger.info("TTS 생성 시작...")

    # 이미지 마커 제거
    clean_text = re.sub(r'\[IMAGE:.*?\]', '', narration).strip()
    if not clean_text:
        clean_text = "대한민국의 놀라운 이야기를 시작합니다."

    # 텍스트 분할 (4000자 제한)

```

```

chunks = []
sentences = re.split(r'(?<=[.!?])\s+', clean_text)
current = ""
for s in sentences:
    if len(current) + len(s) + 1 > 3800:
        if current:
            chunks.append(current.strip())
            current = s
        else:
            current += " " + s if current else s
if current:
    chunks.append(current.strip())

if not chunks:
    chunks = [clean_text[:3800]]

logger.info(f"TTS: {len(chunks)}개 청크")

# 각 청크 TTS 생성
chunk_files = []
for i, chunk in enumerate(chunks):
    chunk_path = AUDIO_DIR / f"{session_id}_chunk_{i:03d}.mp3"
    resp = openai_request("audio/speech", {
        "model": "tts-1-hd",
        "input": chunk,
        "voice": "onyx",
        "speed": 0.92,
        "response_format": "mp3"
    })
    chunk_path.write_bytes(resp.content)
    chunk_files.append(chunk_path)
    logger.info(f"  청크 {i+1}/{len(chunks)} 완료")
    time.sleep(0.5)

# 무음 생성
silence = AUDIO_DIR / f"{session_id}_silence.mp3"
subprocess.run([
    "ffmpeg", "-y", "-f", "lavfi", "-i", "anullsrc=r=44100:cl=mono",
    "-t", "0.4", "-q:a", "9", str(silence)
], capture_output=True)

# 합치기
concat_file = AUDIO_DIR / f"{session_id}_concat.txt"
with open(concat_file, 'w') as f:
    for i, cf in enumerate(chunk_files):
        f.write(f"file '{cf}'\n")
    if i < len(chunk_files) - 1:

```

```

        f.write(f"file '{silence}'\n")

raw_audio = AUDIO_DIR / f"{session_id}_raw.mp3"
subprocess.run([
    "ffmpeg", "-y", "-f", "concat", "-safe", "0",
    "-i", str(concat_file), "-c", "copy", str(raw_audio)
], capture_output=True)

# 음량 정규화
final_audio = AUDIO_DIR / f"{session_id}_final.mp3"
subprocess.run([
    "ffmpeg", "-y", "-i", str(raw_audio),
    "-af", "loudnorm=I=-16:TP=-1.5:LRA=11",
    str(final_audio)
], capture_output=True)

# 정리
for f in chunk_files:
    f.unlink(missing_ok=True)
silence.unlink(missing_ok=True)
concat_file.unlink(missing_ok=True)
raw_audio.unlink(missing_ok=True)

logger.info(f"TTS 완료: {final_audio}")
return final_audio

# =====
# 모듈 2: 이미지 수집
# =====
def collect_images(image_queries: list, session_id: str) -> list:
    """Pexels API로 이미지 수집"""
    logger.info(f"이미지 수집 시작: {len(image_queries)}개 쿼리")

    session_dir = IMAGE_DIR / session_id
    session_dir.mkdir(exist_ok=True)
    downloaded = []

    for i, query in enumerate(image_queries[:25]):
        try:
            resp = requests.get(
                "https://api.pexels.com/v1/search",
                headers={"Authorization": PEXELS_KEY},
                params={"query": query, "per_page": 1, "orientation": "landscape"},
                timeout=10
            )
            resp.raise_for_status()

```

```

        photos = resp.json().get("photos", [])

        if photos:
            img_url = photos[0]["src"]["large2x"]
            img_resp = requests.get(img_url, timeout=30)
            img_path = session_dir / f"img_{i:03d}.jpg"
            img_path.write_bytes(img_resp.content)
            downloaded.append(str(img_path))
            logger.info(f" 이미지 {i+1}: {query[:25]}... ✓")
        else:
            logger.warning(f" 이미지 {i+1}: {query[:25]}... 결과없음")
    except Exception as e:
        logger.warning(f" 이미지 {i+1} 실패: {e}")
    time.sleep(0.3)

# 최소 5개 이미지 보장 (기본 이미지 생성)
while len(downloaded) < 5:
    fallback = session_dir / f"img_fallback_{len(downloaded):03d}.jpg"
    subprocess.run([
        "ffmpeg", "-y", "-f", "lavfi", "-i",
        f"color=c=0x1a1a2e:s=1920x1080:d=1",
        "-frames:v", "1", str(fallback)
    ], capture_output=True)
    downloaded.append(str(fallback))

logger.info(f"이미지 수집 완료: {len(downloaded)}개")
return downloaded

# =====
# 모듈 3: 썸네일 생성
# =====
def generate_thumbnail(thumbnail_text: str, bg_image: str, session_id: str) -> Pa
    """FFmpeg로 썸네일 생성"""
    output = THUMB_DIR / f"{session_id}_thumb.jpg"

    try:
        from PIL import Image, ImageDraw, ImageFont

        if bg_image and os.path.exists(bg_image):
            bg = Image.open(bg_image).resize((1280, 720), Image.LANCZOS).convert(
        else:
            bg = Image.new('RGBA', (1280, 720), (26, 26, 46, 255))

        # 어두운 오버레이
        overlay = Image.new('RGBA', (1280, 720), (0, 0, 0, 150))
        bg = Image.alpha_composite(bg, overlay)

```

```

draw = ImageDraw.Draw(bg)

# 폰트 찾기
font_paths = [
    "/usr/share/fonts/truetype/nanum/NanumGothicBold.ttf",
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
]
font_path = next((p for p in font_paths if os.path.exists(p)), None)
font = ImageFont.truetype(font_path, 64) if font_path else ImageFont.load

# 텍스트 렌더링
lines = thumbnail_text.split('\n') if '\n' in thumbnail_text else [thumbn
y = 250
for line in lines[:2]:
    bbox = draw.textbbox((0, 0), line, font=font)
    tw = bbox[2] - bbox[0]
    th = bbox[3] - bbox[1]
    x = (1280 - tw) // 2
    draw.rectangle([x-12, y-8, x+tw+12, y+th+8], fill='#FF0000')
    draw.text((x, y), line, fill='white', font=font)
    y += th + 30

bg.convert('RGB').save(str(output), "JPEG", quality=95)
except ImportError:
    # Pillow 없으면 FFmpeg fallback
    subprocess.run([
        "ffmpeg", "-y", "-f", "lavfi", "-i",
        "color=c=0x1a1a2e:s=1280x720:d=1",
        "-frames:v", "1", str(output)
    ], capture_output=True)

logger.info(f"썸네일 생성 완료: {output}")
return output

# =====
# 모듈 4: 영상 편집 (FFmpeg)
# =====
def compose_video(images: list, audio_path: Path, narration: str, session_id: str
    """FFmpeg로 이미지 + 오디오 → 영상 합성"""
    logger.info("영상 편집 시작...")

# 오디오 길이
result = subprocess.run([
    "ffprobe", "-v", "quiet", "-show_entries", "format=duration",
    "-of", "csv=p=0", str(audio_path)
], capture_output=True, text=True)

```

```
audio_duration = float(result.stdout.strip()) if result.stdout.strip() else 3
```

```
# 클립 생성
```

```
clip_duration = max(audio_duration / len(images), 3.0)
```

```
effects = ["zoom_in", "zoom_out", "pan_right", "pan_left"]
```

```
zoompan_map = {
```

```
    "zoom_in": "zoompan=z='min(zoom+0.0015,1.3)':d={dur}:s=1920x1080:fps=30",
```

```
    "zoom_out": "zoompan=z='if(eq(on,1),1.3,max(zoom-0.0015,1))':d={dur}:s=19
```

```
    "pan_right": "zoompan=z='1.2':x='if(eq(on,1),0,min(x+2,iw-iw/zoom))':d={d
```

```
    "pan_left": "zoompan=z='1.2':x='if(eq(on,1),iw,max(x-2,0))':d={dur}:s=192
```

```
}
```

```
clip_files = []
```

```
for i, img in enumerate(images):
```

```
    actual_dur = min(clip_duration, audio_duration - (i * clip_duration))
```

```
    if actual_dur <= 0:
```

```
        break
```

```
    clip = VIDEO_DIR / f"{session_id}_clip_{i:03d}.mp4"
```

```
    effect = effects[i % 4]
```

```
    dur_frames = int(actual_dur * 30)
```

```
    zp = zoompan_map[effect].format(dur=dur_frames)
```

```
    subprocess.run([
```

```
        "ffmpeg", "-y", "-loop", "1", "-i", img,
```

```
        "-vf", f"scale=1920:1080:force_original_aspect_ratio=increase,crop=19
```

```
        "-t", str(actual_dur), "-c:v", "libx264", "-pix_fmt", "yuv420p", "-r"
```

```
        str(clip)
```

```
    ], capture_output=True)
```

```
    clip_files.append(clip)
```

```
    logger.info(f" 클립 {i+1}/{len(images)} ({effect})")
```

```
# 클립 이어붙이기
```

```
concat_list = VIDEO_DIR / f"{session_id}_concat.txt"
```

```
with open(concat_list, 'w') as f:
```

```
    for cf in clip_files:
```

```
        f.write(f"file '{cf}'\n")
```

```
raw_video = VIDEO_DIR / f"{session_id}_raw.mp4"
```

```
subprocess.run([
```

```
    "ffmpeg", "-y", "-f", "concat", "-safe", "0",
```

```
    "-i", str(concat_list), "-c:v", "libx264", "-pix_fmt", "yuv420p",
```

```
    str(raw_video)
```

```
], capture_output=True)
```

```
# SRT 자막 생성
```

```
srt_path = VIDEO_DIR / f"{session_id}_subs.srt"
```

```

clean_narr = re.sub(r'\[IMAGE:.*?\]', '', narration).strip()
sentences = [s.strip() for s in re.split(r'(?<=[.!?])\s+', clean_narr) if s.s
total_chars = max(sum(len(s) for s in sentences), 1)
cur_time = 0.0

with open(srt_path, 'w', encoding='utf-8') as f:
    for idx, sent in enumerate(sentences):
        dur = max((len(sent) / total_chars) * audio_duration, 1.0)
        start = cur_time
        end = min(cur_time + dur, audio_duration)
        sh, sm, ss, sms = int(start//3600), int((start%3600)//60), int(start%
        eh, em, es, ems = int(end//3600), int((end%3600)//60), int(end%60), i

        display = sent
        if len(sent) > 30:
            mid = sent.find(' ', len(sent)//2)
            if mid > 0:
                display = sent[:mid] + '\n' + sent[mid+1:]

        f.write(f"{idx+1}\n")
        f.write(f"{sh:02d}:{sm:02d}:{ss:02d},{sms:03d} --> {eh:02d}:{em:02d}:
        f.write(f"{display}\n\n")
        cur_time = end

# BGM 생성
bgm = AUDIO_DIR / f"{session_id}_bgm.mp3"
subprocess.run([
    "ffmpeg", "-y", "-f", "lavfi", "-i",
    f"sine=frequency=220:sample_rate=44100:duration={audio_duration},"
    f"tremolo=f=0.5:d=0.7,lowpass=f=300,volume=0.03",
    "-t", str(audio_duration), str(bgm)
], capture_output=True)

# 오디오 믹싱
mixed = AUDIO_DIR / f"{session_id}_mixed.mp3"
subprocess.run([
    "ffmpeg", "-y", "-i", str(audio_path), "-i", str(bgm),
    "-filter_complex", "[1]volume=0.08[bg];[0][bg]amix=inputs=2:duration=firs
    str(mixed)
], capture_output=True)

# 최종 합성
final = VIDEO_DIR / f"{session_id}_final.mp4"
sub_style = "FontSize=22,Bold=1,PrimaryColour=&H00FFFFFF,OutlineColour=&H0000

subprocess.run([
    "ffmpeg", "-y", "-i", str(raw_video), "-i", str(mixed),

```



```

        "-vf", f"subtitles={srt_path}:force_style='{sub_style}'",
        "-c:v", "libx264", "-preset", "medium", "-crf", "23",
        "-c:a", "aac", "-b:a", "192k",
        "-shortest", "-movflags", "+faststart",
        str(final)
    ], capture_output=True)

# 정리
for cf in clip_files:
    cf.unlink(missing_ok=True)
concat_list.unlink(missing_ok=True)
raw_video.unlink(missing_ok=True)
bgm.unlink(missing_ok=True)
mixed.unlink(missing_ok=True)

size_mb = final.stat().st_size / (1024*1024) if final.exists() else 0
logger.info(f"영상 완성: {final} ({size_mb:.1f}MB)")
return final

# =====
# 전체 파이프라인
# =====
def run_pipeline(script_json: dict):
    """Make.com에서 받은 대본 JSON으로 전체 파이프라인 실행"""
    session_id = datetime.now().strftime("%Y%m%d_%H%M%S")
    notify(f"🎬 <b>파이프라인 시작</b>")

    try:
        title = script_json.get("title", "제목없음")
        narration = script_json.get("narration", "")
        thumbnail_text = script_json.get("thumbnail_text", title)
        image_queries = script_json.get("image_queries", [])
        tags = script_json.get("tags", [])

        notify(f"📝 제목: {title}")

        # 1. TTS
        audio_path = generate_tts(narration, session_id)
        notify(f"🔊 TTS 완료")

        # 2. 이미지 수집
        images = collect_images(image_queries, session_id)
        notify(f"🖼️ 이미지 {len(images)}개 수집")

        # 3. 썸네일
        thumb_path = generate_thumbnail(thumbnail_text, images[0] if images else

```

```

notify("📁 썸네일 완료")

# 4. 영상 편집
video_path = compose_video(images, audio_path, narration, session_id)
notify("🎬 영상 편집 완료")

# 5. YouTube 업로드 (OAuth 설정 후 활성화)
# upload_result = upload_to_youtube(video_path, thumb_path, script_json)

notify(
    f"✅ <b>파이프라인 완료!</b>\n"
    f"제목: {title}\n"
    f"영상: {video_path}\n"
    f"세션: {session_id}"
)

return {
    "status": "success",
    "session_id": session_id,
    "title": title,
    "video_path": str(video_path),
    "thumbnail_path": str(thumb_path),
}

except Exception as e:
    logger.error(f"파이프라인 오류: {e}", exc_info=True)
    notify(f"❌ <b>오류</b>\n{str(e)[:200]}")
    return {"status": "error", "error": str(e)}

# =====
# Flask 엔드포인트
# =====
@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "service": "국뽕유튜브 자동화 서버",
        "status": "running",
        "pipeline_running": pipeline_status["running"],
        "last_run": pipeline_status["last_run"]
    })

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok", "timestamp": datetime.now().isoformat()})

```

```

@app.route("/trigger", methods=["POST"])
def trigger():
    """Make.com에서 호출 - 대본 JSON을 받아 파이프라인 실행"""
    if pipeline_status["running"]:
        return jsonify({"status": "busy", "message": "이미 실행 중"}), 429

    data = request.json or {}
    script_json = data.get("script", data)

    thread = threading.Thread(target=_run_async, args=(script_json,))
    thread.start()

    return jsonify({"status": "started", "timestamp": datetime.now().isoformat()})

@app.route("/status", methods=["GET"])
def status():
    return jsonify(pipeline_status)

def _run_async(script_json):
    pipeline_status["running"] = True
    pipeline_status["last_run"] = datetime.now().isoformat()
    try:
        result = run_pipeline(script_json)
        pipeline_status["last_result"] = result
    except Exception as e:
        pipeline_status["last_result"] = {"status": "error", "error": str(e)}
    finally:
        pipeline_status["running"] = False

# =====
# 실행
# =====
if __name__ == "__main__":
    port = int(os.getenv("PORT", 10000))
    logger.info(f"🎬 서버 시작: http://0.0.0.0:{port}")
    app.run(host="0.0.0.0", port=port)

```